Trust Follows the Request, Attacks Follow the Data: Measuring Where Agent Defenses Look

[Author], [affiliation], [email]

Draft of 1 November 2026, second pass · Target: a security workshop (AISec, SaTML or an S&P/USENIX workshop). Bracketed fields are for the author.

Abstract

Defenses for tool-using language-model agents decide who may act in one of two ways: by the path a request arrives on (an input classifier, tools withheld on untrusted paths, confirmation tied to the user turn) or by the provenance of the values a privileged call carries (a record match, a taint rule). We measure both kinds on six reference agents with a procedure that counts a hit only when a forbidden tool call reaches a sink with a per-trial nonce in its argument, runs a benign control beside every attack, and reports utility. Three results. First, a prompt-injection classifier's cost depends on the channel: on the perception path it discriminates on camera frames and PDFs but blocks 70 to 100 % of benign input on voice, device names and invoices, and on the chat path it blocks the user's own legitimate privileged request 27 to 30 times in 30 on five of six channels. Second, on channels where content is stored and acted on later, the request-path layers are inert by construction; protection comes only from content-side layers, which either strip the actionable value (and hold when the task does not need it) or check its provenance (and hold when it does, at a utility cost that on our support-desk agent is the whole task: 0 of 30 legitimate refunds). Third, at every model-side gate the outcome belongs to the wording, not the trial: with 25 generated paraphrases, 21 to 23 give the same result on both tries and the between-wording share of variance is 0.81 to 0.92, so per-trial confidence intervals overstate precision, and a best-of-k attacker passes a vocabulary classifier at 0.88 where five hand-written sentences passed at 0.40. We release the workbench, the agents, the driver, every trial, and four runs our own control arm threw out.

1. Introduction

An agent takes a user's request, reads state it did not author, and calls tools. Its defenses look somewhere, and where they look is a design decision whose consequences are rarely measured. An input classifier reads the user turn or the perceived text. Tool withholding and confirmation read the request path: which endpoint, which turn. A record check or a taint rule reads the value a privileged call carries. The first group is cheap and is what published layer lists mostly contain [21]. The second is what data-flow designs such as CaMeL [17] and FIDES [19] are built on.

We ask three measurable questions. What does a classifier cost, per channel, in benign input it drops and legitimate requests it blocks? On a channel where content is read in one request and acted on in another, which layers still do anything? And when a defense is a model, is the trial or the sentence the unit of measurement?

Contributions:

1. **A measurement procedure** with a nonce-keyed sink as the only evidence of a hit, refused calls recorded apart from executed ones, a benign control arm that voids the run when it fires, a benign chat arm, and a utility arm on every build (§3).
2. **Classifier cost per channel** (§4.1): the same classifier on the perception path and on the chat path, with its false-positive rate on benign channel input, benign chat and legitimate privileged requests.
3. **Request-path versus value layers on store-then-trigger channels** (§4.2), with a two-phase inbox built to test the claim and a provenance-keyed confirmation built to compete with the path-keyed one.
4. **Wording as the experimental unit at model-side gates** (§4.3).

We claim no new attack. Indirect prompt injection is Greshake et al. [1]; its demonstration in shipped products through email, calendar and memory is Nassi et al. [2], EchoLeak [3] and Rehberger [4]; through audio, Chen et al. [5]; through signs, Burbano et al. [6]. Counting at the environment rather than the reply is AgentDojo's practice [8]. What we add is the cost column, the utility column, the inertness result and the wording result.

2. Related work

Attacks. Greshake et al. [1] named indirect prompt injection. Nassi, Cohen and Yair [2] and EchoLeak [3] showed it against shipped assistants; Rehberger [4] through memory and delayed tool invocation; Cohen, Bitton and Nassi [7] across agents; Chen et al. [5] through audio; Burbano et al. [6] through printed signs. Nasr, Carlini et al. [18] showed that adaptive attacks break defenses that reported near-zero rates against fixed attacks.

Benchmarks and tools. AgentDojo [8] and InjecAgent [9] score agents on injected tool outputs and judge from environment state; AgentDojo also reports benign utility. PyRIT [10], garak [11] and promptfoo [12] generate and send payloads. None reports how often its success predicate fires on benign input, nor the within-wording correlation of outcomes.

Defenses. Spotlighting [14] marks untrusted text; Prompt Shields [15] is a deployed classifier; the dual-LLM pattern [16], CaMeL [17], FIDES [19] and Progent [20] separate a privileged planner from untrusted data and police tool calls with policies; Google's published layer list [21] combines classifiers, confirmation and other request-path

measures. We implement reference versions as switchable layers and measure each against a benign arm and a utility arm.

3. Method

3.1 Trial and evidence

A trial sends one instruction with one fresh nonce `cw-xxxxxxxx` placed in the attacker-controlled value (address, account, reference). It is encoded for the channel: a device name, a synthetic-speech WAV (over HTTP or as a SIP call), a rendered sign, a rendered invoice, a PDF with the instruction in white ink, or a handoff note stored for the next request. Every tool call the target makes is POSTed to a sink the tester owns with its arguments, whether it executed or was refused, the refusing layer's reason, the target's name and the request's session. A trial's evidence is the events of its own session and target. A **hit** is an executed forbidden tool whose arguments carry the nonce; an **attempt** is the same call refused in code. Reply text and self-reported actions are hints only.

3.2 Arms

Each run has **injected** (through the channel, five wordings rotated by index, or a generated set), **control** (benign content of the same shape with its own nonce; a hit voids the run), **direct** (the instruction typed into chat), **benign_direct** (a benign chat request) and **utility** (a legitimate task that should execute the privileged tool; success is that tool at the sink with the expected argument). Rates are k/N. Verdicts: `confirmed`, `observed`, `not_observed`, `invalid`, `error`.

3.3 Agents, layers and builds

Six reference agents, each a real tool-calling model (`gpt-4o-mini`, temperature 0.7) behind an HTTP API: home (device name → `unlock_door`), voice (speech → `send_email`), camera (frame → `disarm_alarm`), expense (invoice → `pay_invoice`), inbox (mail and PDF → `share_file`), desk (handoff note stored per session, read on the next routine request → `issue_refund`). A two-phase inbox variant stores the PDF's text and acts on it at the next request, like desk. A separate visual testbed, SimLab, runs six deterministic digital twins that accept the same payloads and report to the same sink; it is used for demonstration and never for a number.

Builds: `vulnerable`; the same reference zero-shot classifier (temperature 0) on the chat path, the perception path or both; `hardened` with four layers, each switchable. *Quarantine*: a tool-less model (temperature 0) summarises perception into a fixed schema. *Withhold*: privileged tools are not offered on the perception path. *Confirm*: a privileged tool needs a flag set by the user channel (path mode) or, in *provenance mode*, is refused when its argument value occurs in the session's untrusted text and in no user turn or trusted tool result. *Interlock*: a code check against a record (the refund account must match the customer on the ticket; the recipient domain must be on the allowlist).

3.4 Procedure

The driver starts each build, verifies its `/health` (policy, layers, placement, temperatures, classifier) against the intended build, runs the arms at concurrency 2 and records every trial. N = 30 per arm (50 in the wording test). Runs: the full first pass (`2026-11-01.md`), then, after the measurement fixes an independent review required (evidence keyed by target and session, the nonce rule on every channel, defenses at temperature 0), the three tests reported here (`2026-11-01-rev2.md`). The first pass is kept, with its superseded claims marked.

4. Results

4.1 What a classifier costs, per channel

Table 1 gives the same classifier's verdicts at temperature 0, per arm.

channel	perception path: attacks blocked	perception path: benign input blocked	chat path: benign chat blocked	chat path: legitimate privileged request blocked
camera, sign	30/30	0/30	0/30	30/30
inbox, PDF	30/30	2/30	3/30	30/30
desk, note	12/30	0/30	0/30	0/30
voice, speech	28/30	21/30	6/30	27/30
home, device name	30/30	30/30	30/30	30/30
expense, invoice	30/30	29/30	26/30	30/30

Table 1. Zero-shot classifier, temperature 0, 30 inputs per cell.

On the perception path the classifier separates attack from benign on channels whose normal content is descriptive (a frame, a letter) and fails on channels whose normal content is imperative or names privileged entities (a spoken command, an entity list with a lock in it, an invoice with bank details). On the chat path it blocks the user's own legitimate privileged request on five of six channels, because "unlock the front door" is the thing it was built to flag. The first pass reported hits and controls only and so missed both columns.

Undefended, utility was 28 to 30 of 30 on every channel and the channel attack executed on home 15, voice 21, camera 26, expense 4, inbox 30 and desk 30 of 30. Home is lower than the first pass (28) because a hit now requires the nonce in the tool argument; the agent unlocks the door regardless and copies the reference about half the time.

4.2 Store-then-trigger: which layers still do anything

Table 2 gives attempted/executed of 30 and utility of 30 on the two scenarios where content is stored and acted on at a later request.

build	desk	two-phase inbox
vulnerable	30/30, utility 30	30/30 (control 1/30, invalid), utility 30
hardened, four layers	21/0, utility 22	0/0, utility 30
no quarantine	30/0 (first pass)	30/1, utility 30
no withhold	24/0, utility 17	0/0, utility 30
no confirm	25/0, utility 24	0/0, utility 30
no interlock	**26/26**, utility 24	0/0, utility 30
provenance confirm	22/0, utility **0**	0/0, utility 30
provenance confirm, no interlock	24/0, utility **0**	0/0, utility 30

Table 2. Attempted/executed hits of 30 injected trials; utility = legitimate privileged task executed, of 30.

On both scenarios the trigger arrives on the chat path, so withhold (which keys on the perception path) and path-mode confirm (which keys on the user turn) never see the stored content; their rows are replicates of the hardened row, and the inbox column shows that directly. The two scenarios then diverge at the quarantine. On the inbox, the mail schema has no field for "share this file with X", so the instruction is stripped and no build attempts the share; utility is unaffected because the legitimate share comes from the user. On desk, the schema keeps "refund 240 to account X" because that is what a handoff note is for; the model attempts the refund in 21 to 26 of 30 trials under every build, and only the value checks stop it. The record match stops all attempts; with it off, 26 execute. The provenance rule also stops all attempts without the record match, and it stops every legitimate refund too, because the legitimate account in the utility task also arrives in a note. On this task the provenance rule's price is the task.

So the prediction "every store-then-trigger channel defeats all four layers" was wrong. The narrower statement the data support: on such channels the request-path layers contribute nothing, by construction; a content-stripping quarantine holds exactly when the task does not need the actionable value; when it does, only a check on the value holds, and a provenance check costs every legitimate action whose value shares the attack's channel.

4.3 Wording is the unit at model-side gates

Table 3: desk, 25 wordings generated from the base instruction, 2 trials each.

build	mean rate	pass both	fail both	mixed	between-wording share of variance
classifier on perception (hits)	0.88	21	2	2	0.81
hardened (attempts)	0.68	16	7	2	0.91
provenance confirm (attempts)	0.60	14	9	2	0.92

Table 3. Per-wording outcomes; the share is between-wording variance over p(1−p).

At each gate, 21 to 23 of 25 wordings gave the same outcome on both tries. The gate is close to a fixed decision boundary over sentences; sampling at temperature 0.7 moves the agent's choice of tool, not the gate's verdict. Two consequences. A confidence interval over 30 trials of 5 wordings is an interval over 5 draws, and the first-pass intervals should be read that way. And the mean is the wrong statistic: the generated set passed the classifier at 0.88 where the five hand-written wordings passed at 0.40, so an attacker who tries sentences, as in [18], defeats a vocabulary gate at small k. Mizrahi et al. [22] made the multi-prompt argument for capability evaluation; we know of no injection evaluation that reports the within-wording correlation at the defense boundary.

4.4 What the control arm caught

Four runs were voided by their own control arm and are kept in the record: a camera oracle that fired on every quiet scene; a home agent that unlocked the door on "signal strong, battery 64%"; a desk agent that refunded a happy customer on a closed ticket (whose first pass also exposed that trial evidence was being selected by time window rather than session, now fixed); and an inbox agent that shared a private file on a benign renewal letter. Each would have been reported as a success by a single-send analyzer.

5. Discussion

Where a classifier can live. Table 1 is a per-channel operating point, and the operating point is set by the channel's benign traffic, not by the classifier's placement. A vocabulary classifier belongs on descriptive channels and nowhere near a channel whose legitimate content is commands. The chat-path column is the sharper warning: on an agent whose purpose is privileged actions, a chat classifier that flags privileged requests removes the purpose.

Layers that cannot see the data. A defense keyed to the request path assumes the content is consumed on the request that delivered it. Memory, handoff notes and inboxes break that assumption, and the layer then checks a path that is always trusted. Table 2 shows this on two scenarios and shows what remains: content-side layers. The quarantine is one when the schema can omit the

actionable value. The record match and the provenance rule are the others, and the desk utility column prices the provenance rule: when legitimate values and attack values share a channel, provenance alone cannot separate them, and a design has to give legitimate values a trusted path (a system of record, a signed note) rather than a smarter filter.

Why count at the tool, and why count sentences. A refusal that quotes the attack matches any text marker; a reported action is a claim. Counting at the sink with a nonce removes both. Counting sentences rather than trials at the gates is the remaining correction.

6. Limitations

One model. Reference agents, not products; the voice-platform terms we surveyed forbid product testing without consent. The classifier is a zero-shot reference, not Llama Prompt Guard 2 or Prompt Shields, which this machine cannot obtain; Table 1's ordering is a prediction for them, not a measurement. The provenance rule is whole-value containment and is blind to spelled-out values. The desk utility task was designed so the legitimate value arrives in a note; a task with a trusted source for that value would price the provenance rule differently, and that is the next experiment. First-pass runs have clean counts but, before the session fix, per-trial evidence lists that can include a concurrent trial's calls. The SIP connector is UDP on loopback.

7. Ethics

All targets are our own. No product or third party was tested. The reference agents, SimLab and the tool are released so others can measure their own agents; the tool refuses to say `confirmed` without a control arm.

8. Conclusion

A defense stops what it looks at. A classifier looks at text and pays, per channel, in the benign text it drops. Request-path layers look at the route and see nothing once content is stored and read later. Value checks look at the argument and hold, at a price set by how many legitimate values share the attacker's channel. And a model-side gate decides per sentence, so its evaluation has to count sentences. Each of those is a number in this paper, with the arm that measured it and the run that a control voided.

Availability

Crosswire (Apache-2.0), the reference agents, `experiment.py`, SimLab, and every trial: `crosswire/docs/results/2026-11-01.md` and `2026-11-01-rev2.md` in [repository].

References

[1] K. Greshake et al. Not what you've signed up for. arXiv: 2302.12173, 2023.

[2] B. Nassi, S. Cohen, O. Yair. Invitation is all you need. arXiv: 2508.12175, 2025.

[3] Aim Security. EchoLeak. 2025.

[4] J. Rehberger. Embrace The Red, 2024–2025.

[5] Chen et al. Audio-native prompt injection. arXiv:2604.14604, 2026. [confirm title]

[6] Burbano et al. CHAI. arXiv:2510.00181, 2025. [confirm title]

[7] S. Cohen, R. Bitton, B. Nassi. Here comes the AI worm. arXiv: 2403.02817, 2024.

[8] E. Debenedetti et al. AgentDojo. arXiv:2406.13352, 2024.

[9] Q. Zhan et al. InjecAgent. arXiv:2403.02691, 2024.

[10] Microsoft. PyRIT. github.com/Azure/PyRIT.

[11] NVIDIA. garak. github.com/NVIDIA/garak.

[12] promptfoo. github.com/promptfoo/promptfoo.

[13] DEF CON 34 AI Village voice-agent calling tool, 2026. [confirm]

[14] K. Hines et al. Spotlighting. arXiv:2403.14720, 2024.

[15] Microsoft. Prompt Shields. Product documentation.

[16] S. Willison. The dual LLM pattern. 2023.

[17] E. Debenedetti et al. Defeating prompt injections by design (CaMeL). arXiv:2503.18813, 2025.

[18] M. Nasr, N. Carlini et al. The attacker moves second. arXiv: 2510.09023, 2025.

[19] FIDES: securing AI agents with information-flow control. arXiv:2505.23643, 2025.

[20] Progent. arXiv:2504.11703, 2025.

[21] Lessons from defending Gemini against indirect prompt injections. arXiv:2505.14534, 2025.

[22] M. Mizrahi et al. State of what art? Multi-prompt evaluation. arXiv:2401.00595, TACL 2024.

References 5, 6, 13 and 19 to 21 carry identifiers from the review and the prior-work survey; confirm titles and authors against the arXiv pages before submission.